Credit Card Fraud Detection Using Deep Learning

This notebook implements MLP, 1D CNN and Autoencoder; BCE, class-weighted BCE and Focal Loss experiments; validation threshold selection; test evaluation; and optional SHAP/LIME explanations.

1. Install packages

```
!pip -q install shap lime
```

```
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Building wheel for lime (pyproject.toml) ... done
```

2. Imports and reproducibility

```
import os, random, warnings
warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (precision_score, recall_score, f1_score,
    average_precision_score, roc_auc_score, confusion_matrix,
    classification_report, precision_recall_curve)

SEED=42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)
print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

```
TensorFlow: 2.21.0
GPU: []
```

3. Upload and load dataset

```
from google.colab import files
import pandas as pd

# Upload the actual creditcard.csv file (not archive.zip)
uploaded = files.upload()

csv_files = [name for name in uploaded if name.lower().endswith('.csv')]
if not csv_files:
    raise ValueError('Please upload creditcard.csv as a .csv file, not a ZIP archive.')

# Prefer the expected dataset filename if more than one CSV is uploaded.
csv_name = next((name for name in csv_files if 'creditcard' in name.lower()), csv_files[0])
df = pd.read_csv(csv_name)

required_columns = {'Class', 'Time', 'Amount'}
missing_columns = required_columns - set(df.columns)
if missing_columns:
    raise ValueError(f'This does not look like the expected credit card dataset. Missing columns: {missing_colum

print('Dataset loaded successfully!')
print('File:', csv_name)
print('Dataset shape:', df.shape)
display(df.head())
```

```
Choose files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving creditcard.csv to creditcard.csv
Dataset loaded successfully!
File: creditcard.csv
Dataset shape: (284807, 31)
```

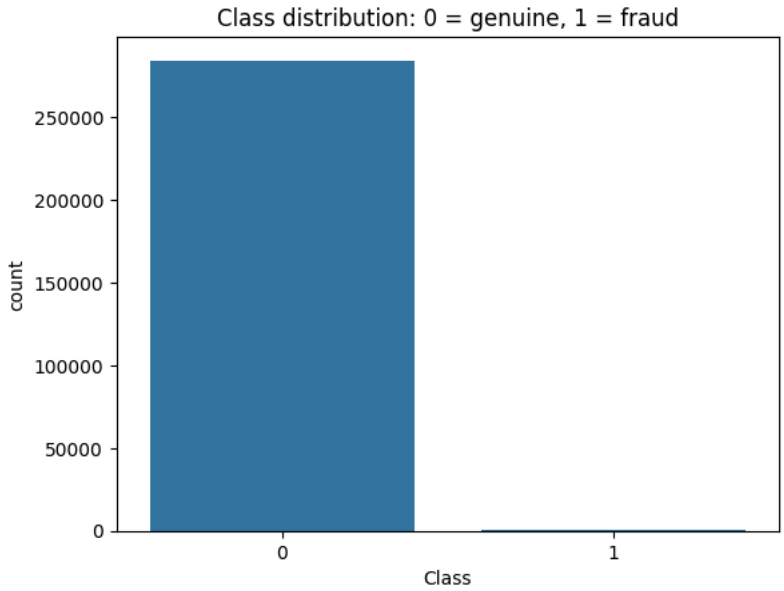
	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278

5 rows × 31 columns

4. Inspect dataset and class imbalance

```
print("Missing values:", df.isnull().sum().sum())
print("Duplicate rows:", df.duplicated().sum())
print(df["Class"].value_counts())
print((df["Class"].value_counts(normalize=True)*100).round(4))
sns.countplot(data=df, x="Class")
plt.title("Class distribution: 0 = genuine, 1 = fraud")
plt.show()
assert "Class" in df.columns and df.isnull().sum().sum()==0
```

```
Missing values: 0
Duplicate rows: 1081
Class
0    284315
1       492
Name: count, dtype: int64
Class
0    99.8273
1     0.1727
Name: proportion, dtype: float64
```



5. Split into training, validation and test sets

```
X=df.drop(columns=["Class"]).copy()
y=df["Class"].astype("int32").copy()
X_train, X_temp, y_train, y_temp=train_test_split(
    X,y,test_size=0.40,stratify=y,random_state=SEED)
X_val, X_test, y_val, y_test=train_test_split(
    X_temp,y_temp,test_size=0.50,stratify=y_temp,random_state=SEED)
print("Train",X_train.shape,"fraud",int(y_train.sum()))
print("Validation",X_val.shape,"fraud",int(y_val.sum()))
print("Test",X_test.shape,"fraud",int(y_test.sum()))
```

```
Train (170884, 30) fraud 295
Validation (56961, 30) fraud 98
```

```
Test (56962, 30) fraud 99
```

6. Scale Time and Amount using training data only

```
feature_names=X_train.columns.tolist()
scaler=StandardScaler()
scale_cols=["Time","Amount"]
X_train.loc[:,scale_cols]=scaler.fit_transform(X_train[scale_cols])
X_val.loc[:,scale_cols]=scaler.transform(X_val[scale_cols])
X_test.loc[:,scale_cols]=scaler.transform(X_test[scale_cols])
X_train_np=X_train.to_numpy(dtype=np.float32)
X_val_np=X_val.to_numpy(dtype=np.float32)
X_test_np=X_test.to_numpy(dtype=np.float32)
y_train_np=y_train.to_numpy(dtype=np.float32)

y_val_np=y_val.to_numpy(dtype=np.int32)
y_test_np=y_test.to_numpy(dtype=np.int32)
print("Input feature count:",X_train_np.shape[1])
```

```
Input feature count: 30
```

7. Calculate class weights

```
w=compute_class_weight(class_weight="balanced",classes=np.array([0,1]),y=y_train_np.astype(int))
class_weights={0:float(w[0]),1:float(w[1])}
print(class_weights)
```

```
{0: 0.5008646512963908, 1: 289.63389830508476}
```

8. Define loss functions and deep learning architectures

```
def focal_loss(gamma=2.0, alpha=0.75):
    def loss(y_true,y_pred):
        yt=tf.cast(tf.reshape(y_true,(-1,1)),tf.float32)
        yp=tf.clip_by_value(tf.cast(y_pred,tf.float32),1e-7,1-1e-7)
        pt=yt*yp+(1-yt)*(1-yp)
        at=yt*alpha+(1-yt)*(1-alpha)
        return tf.reduce_mean(-at*tf.pow(1-pt,gamma)*tf.math.log(pt))
    return loss

def build_mlp(n):
    return tf.keras.Sequential([
        tf.keras.layers.Input((n,)),
        tf.keras.layers.Dense(64,activation="relu"),
        tf.keras.layers.Dropout(.30),
        tf.keras.layers.Dense(32,activation="relu"),
        tf.keras.layers.Dropout(.20),
        tf.keras.layers.Dense(1,activation="sigmoid")])

def build_cnn(shape):
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape),
        tf.keras.layers.Conv1D(64,3,padding="same",activation="relu"),
        tf.keras.layers.MaxPooling1D(2),
        tf.keras.layers.Conv1D(32,3,padding="same",activation="relu"),
        tf.keras.layers.GlobalMaxPooling1D(),
        tf.keras.layers.Dense(32,activation="relu"),
        tf.keras.layers.Dropout(.30),
        tf.keras.layers.Dense(1,activation="sigmoid")])

def compile_model(model,loss="binary_crossentropy"):
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),loss=loss)
    return model

def get_callbacks():
    return [tf.keras.callbacks.EarlyStopping(monitor="val_loss",patience=3,restore_best_weights=True)]

EPOCHS=15
BATCH_SIZE=512
```

9. Train MLP with standard Binary Cross-Entropy (baseline)

```
mlp_bce=compile_model(build_mlp(X_train_np.shape[1]))
hist_mlp_bce=mlp_bce.fit(X_train_np,y_train_np,validation_data=(X_val_np,y_val_np),
    epochs=EPOCHS,batch_size=BATCH_SIZE,callbacks=get_callbacks(),verbose=1)
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.0582 - val_loss: 0.0057
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0051 - val_loss: 0.0052
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0042 - val_loss: 0.0042
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0039 - val_loss: 0.0039
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0036 - val_loss: 0.0039
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0033 - val_loss: 0.0037
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0030 - val_loss: 0.0037
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0028 - val_loss: 0.0037
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0029 - val_loss: 0.0036
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0029 - val_loss: 0.0037
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0027 - val_loss: 0.0036
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0027 - val_loss: 0.0036
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0025 - val_loss: 0.0037
Epoch 14/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0023 - val_loss: 0.0036
Epoch 15/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0023 - val_loss: 0.0037
```

10. Train MLP with class-weighted BCE

```
mlp_weighted=compile_model(build_mlp(X_train_np.shape[1]))
hist_mlp_weighted=mlp_weighted.fit(X_train_np,y_train_np,validation_data=(X_val_np,y_val_np),
    epochs=EPOCHS,batch_size=BATCH_SIZE,class_weight=class_weights,
    callbacks=get_callbacks(),verbose=1)
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2938 - val_loss: 0.1198
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1579 - val_loss: 0.0988
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.1503 - val_loss: 0.1091
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1206 - val_loss: 0.0933
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1229 - val_loss: 0.0932
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1134 - val_loss: 0.0736
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1002 - val_loss: 0.0656
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1104 - val_loss: 0.0844
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1119 - val_loss: 0.0908
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0921 - val_loss: 0.0676
```

11. Train MLP with Focal Loss

```
mlp_focal=compile_model(build_mlp(X_train_np.shape[1]),focal_loss())
hist_mlp_focal=mlp_focal.fit(X_train_np,y_train_np,validation_data=(X_val_np,y_val_np),
    epochs=EPOCHS,batch_size=BATCH_SIZE,callbacks=get_callbacks(),verbose=1)
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.0031 - val_loss: 0.0012
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 9.7598e-04 - val_loss: 9.0140e-04
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 8.0475e-04 - val_loss: 8.1111e-04
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 6.8384e-04 - val_loss: 7.6327e-04
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 6.1602e-04 - val_loss: 7.1912e-04
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 6.0302e-04 - val_loss: 6.7074e-04
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 5.2429e-04 - val_loss: 6.4087e-04
```

```
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 5.1127e-04 - val_loss: 6.3897e-04
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 5.0453e-04 - val_loss: 6.5698e-04
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 5.0404e-04 - val_loss: 6.3610e-04
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 4.5670e-04 - val_loss: 6.5639e-04
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 4.2796e-04 - val_loss: 6.7317e-04
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 3.7234e-04 - val_loss: 6.9298e-04
```

12. Prepare CNN input and train CNN with BCE

```
X_train_cnn=X_train_np.reshape(-1,X_train_np.shape[1],1)
X_val_cnn=X_val_np.reshape(-1,X_val_np.shape[1],1)
X_test_cnn=X_test_np.reshape(-1,X_test_np.shape[1],1)
cnn_bce=compile_model(build_cnn((X_train_cnn.shape[1],1)))
hist_cnn_bce=cnn_bce.fit(X_train_cnn,y_train_np,validation_data=(X_val_cnn,y_val_np),
    epochs=EPOCHS,batch_size=BATCH_SIZE,callbacks=get_callbacks(),verbose=1)
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 9s 21ms/step - loss: 0.0382 - val_loss: 0.0095
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.0072 - val_loss: 0.0059
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - loss: 0.0050 - val_loss: 0.0046
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.0042 - val_loss: 0.0045
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - loss: 0.0040 - val_loss: 0.0043
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.0037 - val_loss: 0.0045
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - loss: 0.0037 - val_loss: 0.0043
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 19ms/step - loss: 0.0036 - val_loss: 0.0042
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 9s 16ms/step - loss: 0.0035 - val_loss: 0.0041
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - loss: 0.0034 - val_loss: 0.0039
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.0033 - val_loss: 0.0039
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - loss: 0.0032 - val_loss: 0.0038
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.0032 - val_loss: 0.0036
Epoch 14/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - loss: 0.0031 - val_loss: 0.0038
Epoch 15/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - loss: 0.0030 - val_loss: 0.0039
```

13. Train CNN with class-weighted BCE

```
cnn_weighted=compile_model(build_cnn((X_train_cnn.shape[1],1)))
hist_cnn_weighted=cnn_weighted.fit(X_train_cnn,y_train_np,validation_data=(X_val_cnn,y_val_np),
    epochs=EPOCHS,batch_size=BATCH_SIZE,class_weight=class_weights,
    callbacks=get_callbacks(),verbose=1)
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 7s 19ms/step - loss: 0.3788 - val_loss: 0.3402
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.2590 - val_loss: 0.2225
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - loss: 0.2457 - val_loss: 0.1933
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 17ms/step - loss: 0.2045 - val_loss: 0.1486
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.1845 - val_loss: 0.1083
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - loss: 0.1814 - val_loss: 0.3377
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step - loss: 0.1717 - val_loss: 0.1363
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 15ms/step - loss: 0.1576 - val_loss: 0.1207
```

14. Train CNN with Focal Loss

```
cnn_focal=compile_model(build_cnn((X_train_cnn.shape[1],1)),focal_loss())
hist_cnn_focal=cnn_focal.fit(X_train_cnn,y_train_np,validation_data=(X_val_cnn,y_val_np),
    epochs=EPOCHS,batch_size=BATCH_SIZE,callbacks=get_callbacks(),verbose=1)
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 29ms/step - loss: 0.0029 - val_loss: 0.0011
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 16ms/step - loss: 9.6899e-04 - val_loss: 8.5817e-04
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - loss: 7.7268e-04 - val_loss: 8.4377e-04
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 17ms/step - loss: 7.3795e-04 - val_loss: 8.0394e-04
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 6.9038e-04 - val_loss: 7.7799e-04
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - loss: 6.3851e-04 - val_loss: 7.9406e-04
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 6.4668e-04 - val_loss: 8.1605e-04
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 17ms/step - loss: 5.9174e-04 - val_loss: 7.2657e-04
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 6.3538e-04 - val_loss: 7.5460e-04
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 17ms/step - loss: 5.7982e-04 - val_loss: 7.6975e-04
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 6s 17ms/step - loss: 5.8190e-04 - val_loss: 7.7660e-04
```

15. Train Autoencoder on genuine transactions

```
inp=tf.keras.layers.Input(shape=(X_train_np.shape[1],))
z=tf.keras.layers.Dense(16,activation="relu")(inp)
z=tf.keras.layers.Dense(8,activation="relu")(z)
z=tf.keras.layers.Dense(16,activation="relu")(z)
out=tf.keras.layers.Dense(X_train_np.shape[1],activation="linear")(z)
autoencoder=tf.keras.Model(inp,out)
autoencoder.compile(optimizer="adam",loss="mse")
X_train_genuine=X_train_np[y_train_np==0]
hist_ae=autoencoder.fit(X_train_genuine,X_train_genuine,
    validation_data=(X_val_np[y_val_np==0],X_val_np[y_val_np==0]),
    epochs=20,batch_size=BATCH_SIZE,callbacks=get_callbacks(),verbose=1)

def reconstruction_errors(model,data):
    rec=model.predict(data,batch_size=BATCH_SIZE,verbose=0)
    return np.mean(np.square(data-rec),axis=1)
ae_val_scores=reconstruction_errors(autoencoder,X_val_np)
ae_test_scores=reconstruction_errors(autoencoder,X_test_np)
```

```
Epoch 1/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.8527 - val_loss: 0.6241
Epoch 2/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.5237 - val_loss: 0.4706
Epoch 3/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.4431 - val_loss: 0.4320
Epoch 4/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.4131 - val_loss: 0.4080
Epoch 5/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3927 - val_loss: 0.3913
Epoch 6/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3774 - val_loss: 0.3787
Epoch 7/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3657 - val_loss: 0.3693
Epoch 8/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3571 - val_loss: 0.3624
Epoch 9/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3510 - val_loss: 0.3571
Epoch 10/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3466 - val_loss: 0.3533
Epoch 11/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3436 - val_loss: 0.3508
Epoch 12/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3413 - val_loss: 0.3490
Epoch 13/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3396 - val_loss: 0.3473
Epoch 14/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3381 - val_loss: 0.3456
Epoch 15/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3367 - val_loss: 0.3441
Epoch 16/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3354 - val_loss: 0.3427
Epoch 17/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3342 - val_loss: 0.3413
Epoch 18/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3329 - val_loss: 0.3395
Epoch 19/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3318 - val_loss: 0.3382
```

```
Epoch 20/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3307 - val_loss: 0.3369
```

16. Optimize decision thresholds using validation predictions only

```
def choose_threshold(y_true,probs,objective="f1",min_precision=None):
    p,r,t=precision_recall_curve(y_true,probs)
    if len(t)==0: return 0.5
    p,r=p[:-1],r[:-1]
    f1=2*p*r/(p+r+1e-12)
    if min_precision is not None:
        eligible=np.where(p>=min_precision)[0]
        if len(eligible): return float(t[eligible[np.argmax(r[eligible])]])
    idx=int(np.argmax(r if objective=="recall" else f1))
    return float(t[idx])

models={
 "MLP - BCE":(mlp_bce,X_val_np,X_test_np),
 "MLP - Weighted BCE":(mlp_weighted,X_val_np,X_test_np),
 "MLP - Focal Loss":(mlp_focal,X_val_np,X_test_np),
 "CNN - BCE":(cnn_bce,X_val_cnn,X_test_cnn),
 "CNN - Weighted BCE":(cnn_weighted,X_val_cnn,X_test_cnn),
 "CNN - Focal Loss":(cnn_focal,X_val_cnn,X_test_cnn)
}
thresholds={}
for name,(model,xv,xt) in models.items():
    vp=model.predict(xv,batch_size=BATCH_SIZE,verbose=0).ravel()
    thresholds[name]=choose_threshold(y_val_np,vp)
thresholds["Autoencoder"]=choose_threshold(y_val_np,ae_val_scores)
print(thresholds)
```

```
{'MLP - BCE': 0.6594208478927612, 'MLP - Weighted BCE': 0.9992520809173584, 'MLP - Focal Loss': 0.541767895221710
```

17. Evaluate every model on the untouched test set

```
def evaluate(name,y_true,probs,threshold):
    pred=(probs>=threshold).astype(int)
    print("\\n",name,"threshold:",round(threshold,6))
    print(classification_report(y_true,pred,digits=4,zero_division=0))
    print("Confusion matrix:\\n",confusion_matrix(y_true,pred))
    return {"Model":name,"Threshold":threshold,
        "Precision":precision_score(y_true,pred,zero_division=0),
        "Recall":recall_score(y_true,pred,zero_division=0),
        "F1":f1_score(y_true,pred,zero_division=0),
        "PR-AUC":average_precision_score(y_true,probs),
        "ROC-AUC":roc_auc_score(y_true,probs)},pred

results=[]; test_probs={}; test_preds={}
for name,(model,xv,xt) in models.items():
    prob=model.predict(xt,batch_size=BATCH_SIZE,verbose=0).ravel()
    test_probs[name]=prob
    row,pred=evaluate(name,y_test_np,prob,thresholds[name])
    results.append(row); test_preds[name]=pred

row,pred=evaluate("Autoencoder",y_test_np,ae_test_scores,thresholds["Autoencoder"])
results.append(row); test_preds["Autoencoder"]=pred
results_df=pd.DataFrame(results).sort_values("PR-AUC",ascending=False)
display(results_df)
```

```
\n MLP - BCE threshold: 0.659421
              precision    recall  f1-score   support

           0     0.9997    0.9998    0.9998     56863
           1     0.8817    0.8283    0.8542        99

    accuracy                         0.9995     56962
   macro avg     0.9407    0.9140    0.9270     56962
weighted avg     0.9995    0.9995    0.9995     56962

Confusion matrix:\n [[56852    11]
 [   17    82]]
\n MLP - Weighted BCE threshold: 0.999252
              precision    recall  f1-score   support

           0     0.9996    0.9998    0.9997     56863
           1     0.8750    0.7778    0.8235        99

    accuracy                         0.9994     56962
   macro avg     0.9373    0.8888    0.9116     56962
weighted avg     0.9994    0.9994    0.9994     56962

Confusion matrix:\n [[56852    11]
 [   22    77]]
\n MLP - Focal Loss threshold: 0.541768
              precision    recall  f1-score   support

           0     0.9996    0.9998    0.9997     56863
           1     0.8778    0.7980    0.8360        99

    accuracy                         0.9995     56962
   macro avg     0.9387    0.8989    0.9179     56962
weighted avg     0.9994    0.9995    0.9994     56962

Confusion matrix:\n [[56852    11]
 [   20    79]]
\n CNN - BCE threshold: 0.507984
              precision    recall  f1-score   support

           0     0.9996    0.9998    0.9997     56863
           1     0.8681    0.7980    0.8316        99

    accuracy                         0.9994     56962
   macro avg     0.9339    0.8989    0.9156     56962
weighted avg     0.9994    0.9994    0.9994     56962

Confusion matrix:\n [[56851    12]
 [   20    79]]
\n CNN - Weighted BCE threshold: 0.990506
              precision    recall  f1-score   support

           0     0.9995    0.9994    0.9994     56863
           1     0.6571    0.6970    0.6765        99

    accuracy                         0.9988     56962
   macro avg     0.8283    0.8482    0.8379     56962
weighted avg     0.9989    0.9988    0.9989     56962

Confusion matrix:\n [[56827    36]
 [   30    69]]
\n CNN - Focal Loss threshold: 0.502777
```

18. Plot confusion matrices and compare metrics

```
for name,pred in test_preds.items():
    plt.figure(figsize=(4,3))
    sns.heatmap(confusion_matrix(y_test_np,pred),annot=True,fmt="d",cmap="Blues")
    plt.title("Confusion Matrix: "+name); plt.xlabel("Predicted"); plt.ylabel("Actual")
    plt.show()

results_df.set_index("Model")[["Precision","Recall","F1","PR-AUC"]].plot(
    kind="bar",figsize=(12,5),ylim=(0,1),rot=35)
plt.title("Test-set comparison"); plt.ylabel("Score"); plt.tight_layout(); plt.show()
```

```
\n Autoencoder threshold: 4.721882
              precision    recall  f1-score   support

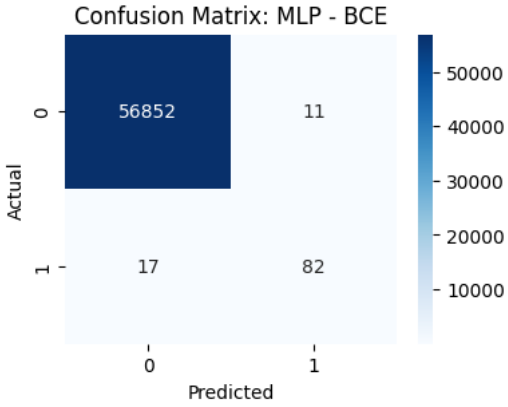
           0     0.9995    0.9984    0.9990     56863
           1     0.4367    0.6970    0.5370        99

    accuracy                         0.9979     56962
   macro avg     0.7181    0.8477    0.7680     56962
weighted avg     0.9985    0.9979    0.9982     56962

Confusion matrix:\n [[56774    89]
 [   30    69]]
```

	Model	Threshold	Precision	Recall	F1	PR-AUC	ROC-AUC
0	MLP - BCE	0.659421	0.881720	0.828283	0.854167	0.843964	0.979354
2	MLP - Focal Loss	0.541768	0.877778	0.797980	0.835979	0.840167	0.978063

3 CNN - BCE 0.507984 0.868132 0.797980 0.831579 0.799615 0.967543

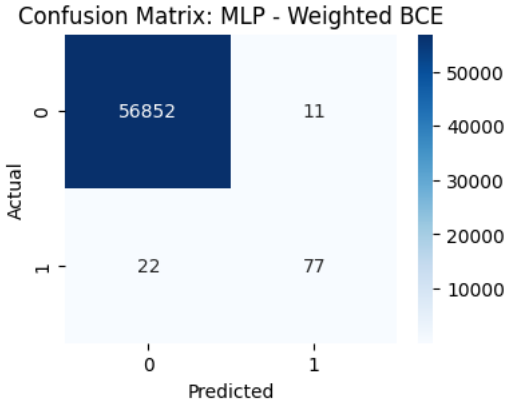


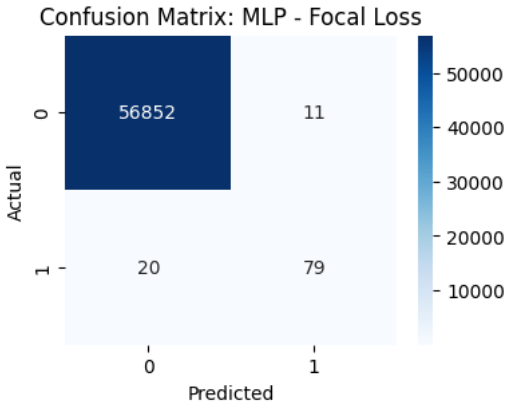
677 0.821622 0.796388 0.970793

778 0.823529 0.726813 0.975387

970 0.676471 0.647397 0.954998

970 0.536965 0.486904 0.961803



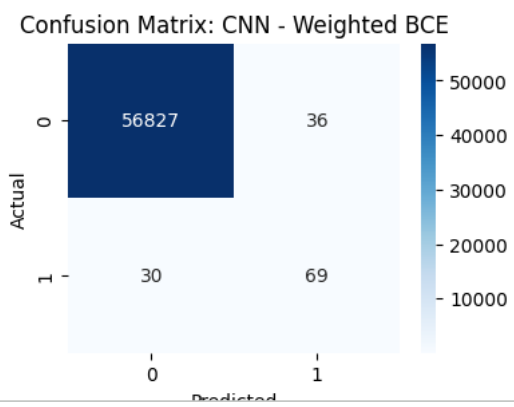


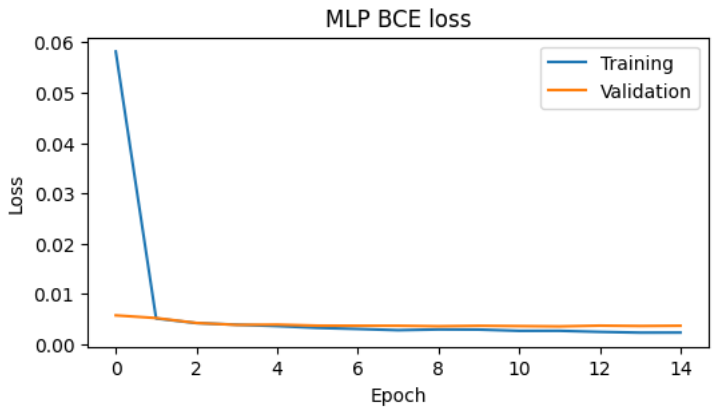


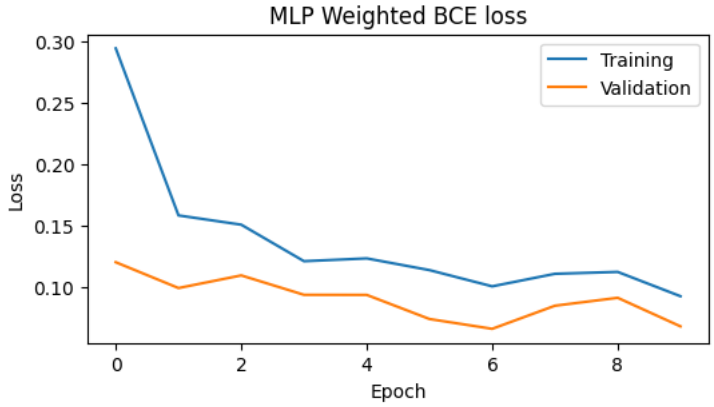
19. Plot training and validation loss

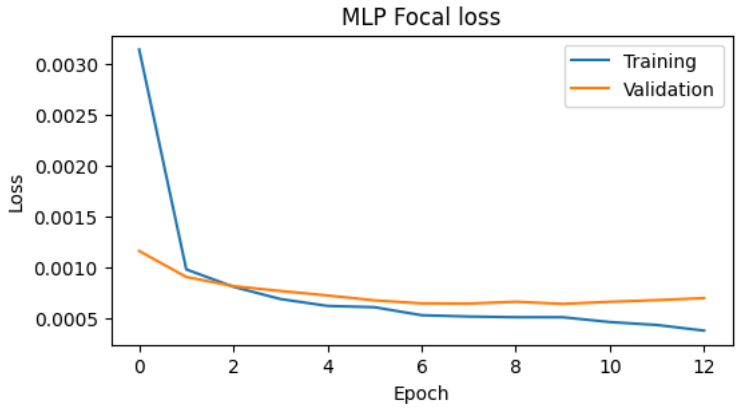
```
histories={"MLP BCE":hist_mlp_bce,"MLP Weighted BCE":hist_mlp_weighted,
 "MLP Focal":hist_mlp_focal,"CNN BCE":hist_cnn_bce,
 "CNN Weighted BCE":hist_cnn_weighted,"CNN Focal":hist_cnn_focal,"Autoencoder":hist_ae}
for name,h in histories.items():
    plt.figure(figsize=(6,3))
    plt.plot(h.history["loss"],label="Training")
    if "val_loss" in h.history: plt.plot(h.history["val_loss"],label="Validation")
    plt.title(name+" loss"); plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.legend(); plt.show()
```

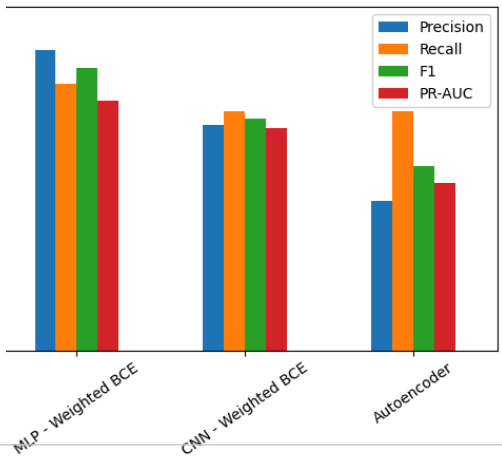
Predicted

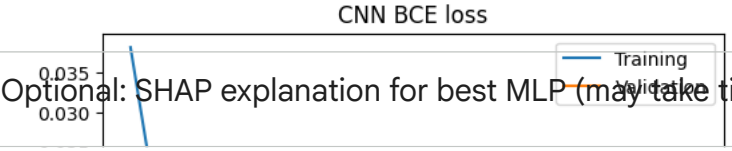






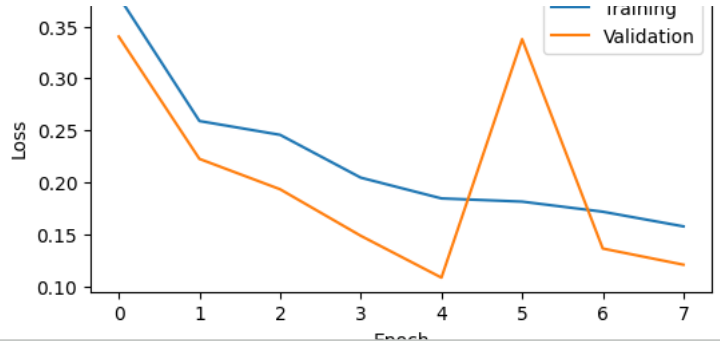




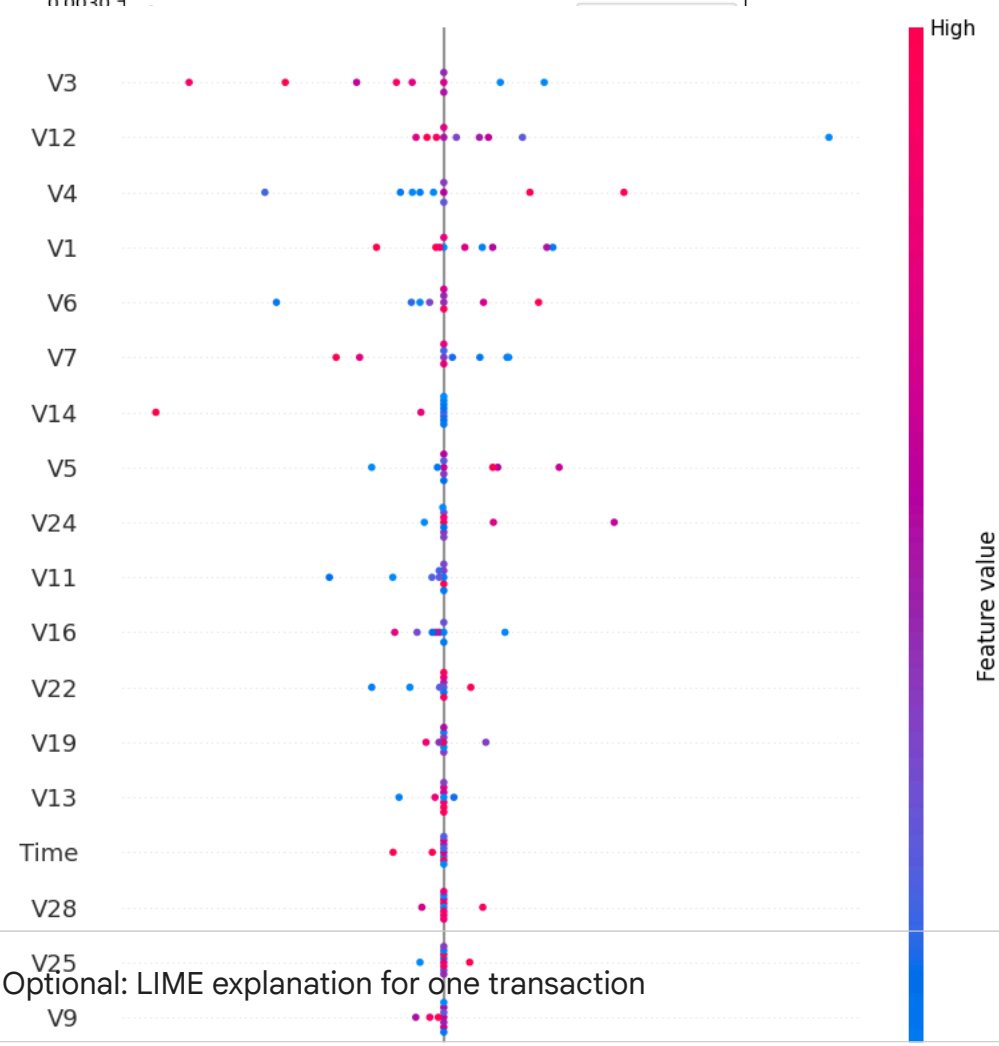


20. Optional: SHAP explanation for best MLP (may take time)

```
import shap
mlp_variants={"MLP - BCE":mlp_bce,"MLP - Weighted BCE":mlp_weighted,"MLP - Focal Loss":mlp_focal}
val_ap={n:average_precision_score(y_val_np,m.predict(X_val_np,verbose=0).ravel())
        for n,m in mlp_variants.items()}
best_mlp_name=max(val_ap,key=val_ap.get); best_mlp=mlp_variants[best_mlp_name]
print("Best MLP on validation PR-AUC:",best_mlp_name)
background=X_train_np[np.random.choice(len(X_train_np),min(40,len(X_train_np)),replace=False)]
explain_data=X_test_np[:10]
explainer=shap.KernelExplainer(lambda a:best_mlp.predict(np.asarray(a,dtype=np.float32),verbose=0).ravel(),backg
shap_values=explainer.shap_values(explain_data,nsamples=100)
sv=shap_values[0] if isinstance(shap_values,list) else shap_values
shap.summary_plot(sv,explain_data,feature_names=feature_names)
```



```
Best MLP on validation PR-AUC: MLP – BCE
```



21. Optional: LIME explanation for one transaction

```
from lime.lime_tabular import LimeTabularExplainer
lime_explainer=LimeTabularExplainer(X_train_np,feature_names=feature_names,
    class_names=["Genuine","Fraud"],mode="classification")
def predict_classes(a):
    p=best_mlp.predict(np.asarray(a,dtype=np.float32),verbose=0).ravel()
    return np.column_stack([1-p,p])
i=0
exp=lime_explainer.explain_instance(X_test_np[i],predict_classes,num_features=10,num_samples=1000)
print("Actual class:",int(y_test_np[i]))
print("Predicted fraud probability:",float(predict_classes(X_test_np[i:i+1])[0,1]))
exp.show_in_notebook(show_table=True)
```

```
Actual class: 0
Predicted fraud probability: 2.4070961899269605e-06
```

